Class 26: MLOps Foundations and Production Readiness

Session 26 of 36 – MLOps Block

Data-driven Systems Engineering

Agenda

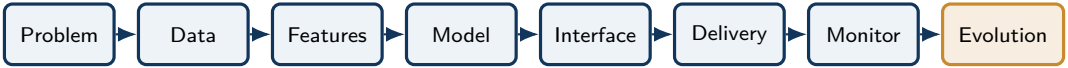
- AI maturity and why adoption is hard
- Overview of MLOps
- Workflow, lifecycle processes, infrastructure, and roles
- Preparing for production
- What maturity looks like in this repository

MLOps is coordination under change

Learning goals

- Define MLOps as the integration of model development with operational discipline.
- Understand the roles and handoffs involved in production ML.
- Evaluate whether a project is production-ready or only demo-ready.
- Distinguish DevOps concerns from additional MLOps concerns.

Course Map and Running Cases



Today in the course

Focus: Operational ownership across the ML lifecycle.
Bridge: This class connects engineering artifacts to production responsibilities, showing how data, models, services, and monitoring become one operating system.

Fraud case

In the fraud case, MLOps means reproducible training, artifact versioning, safe deployment, and monitoring for score shift and analyst load.

Pasteurization case

In the pasteurization case, MLOps means streaming behavior, drift detection, rollback thinking, and operational visibility for process anomalies.

Course thread: This is the point where the course unifies software engineering, experimentation, delivery, and monitoring into one lifecycle.

Why MLOps became necessary

- Training a model is only one part of delivering value.
- Organizations discovered that models fail at the boundaries: data access, packaging, deployment, monitoring, and ownership.
- MLOps formalizes the work needed to make ML systems reproducible, operable, and governable.

Core idea

MLOps is not a single tool. It is a coordination discipline supported by automation, standards, and clear responsibility.

Why AI adoption is hard

- Excessive manual work across teams.
- Fragile handoffs from data engineering to modeling to deployment.
- Missing standards for experiments, artifacts, and rollback.
- Poor visibility into model behavior after release.

Common failure points in ML adoption

- A model works in a notebook but cannot be rebuilt consistently.
- Data and feature assumptions are undocumented.
- Deployment ownership is unclear between data, software, and ops teams.
- Monitoring starts after launch instead of before launch.
- Governance appears only when something has already gone wrong.

AI maturity view

- Early stage: isolated experiments and manual deployment.
- Emerging stage: repeatable training and some CI/CD.
- Managed stage: monitoring, governance, and clearer role boundaries.
- Scaled stage: standardized pipelines, traceability, and controlled retraining.

What maturity changes in practice

Maturity signal	What it means
Repeatable experiments	Teams can reproduce a result from code, data, and config.
Versioned artifacts	A deployed model can be traced and rolled back.
Automated delivery	Packaging and validation happen consistently.
Monitoring and alerts	Teams detect failure before users must explain it.
Governed retraining	Change happens through a controlled path, not by accident.

Where MLOps adds work beyond DevOps

- Data versioning and validation.
- Experiment tracking and model registry.
- Training, retraining, and rollback logic.
- Monitoring for data, model, and business behavior.
- Governance for fairness, auditability, and safe access.

DevOps versus MLOps

DevOps focus

- Build and release software safely
- Reliability, scaling, observability
- Infrastructure and deployment automation

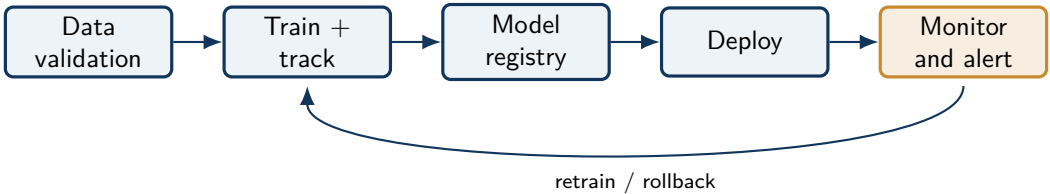
MLOps additions

- Data and feature validation
- Training and retraining workflows
- Model registry, drift, and evaluation governance

A practical MLOps lifecycle

1. Frame the business question and success criteria.
2. Acquire, validate, and document data.
3. Train and compare candidate models.
4. Package the winning artifact and dependencies.
5. Validate through tests and release gates.
6. Deploy with observability.
7. Monitor, diagnose, and decide on retraining or rollback.

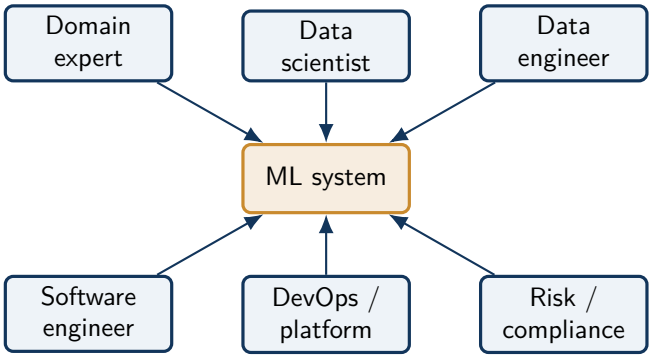
MLOps workflow



Infrastructure layers behind the workflow

- Storage layer: raw data, features, models, logs.
- Compute layer: notebooks, training jobs, CI runners, containers.
- Serving layer: APIs, dashboards, schedulers, or streaming components.
- Observability layer: metrics, logs, drift detection, alerts, dashboards.
- Governance layer: permissions, approvals, lineage, and audit records.

Roles around the lifecycle



- MLOps is mostly about reducing friction across these handoffs.

Typical handoffs that create friction

- Data engineering to data science: access, freshness, schema meaning.
- Data science to software engineering: artifact packaging, interfaces, dependencies.
- Software engineering to operations: deployment behavior, scaling, secrets, rollback.
- Operations back to the ML team: incidents, drift, and degraded outcomes.

Lesson

If a team cannot explain its handoffs, it is probably not ready for dependable production work.

A practical production-readiness checklist

- Can we rebuild the model from code, data, and config?
- Can we explain which version is running and why?
- Do we know how the service fails and who is notified?
- Is there a monitoring plan before launch?
- Do stakeholders agree on success and rollback criteria?

Demo-ready versus production-ready

Demo-ready

- Works on one machine
- Manual execution
- Little failure handling
- Monitoring absent or superficial

Production-ready

- Reproducible environment
- Versioned artifacts and configs
- Clear ownership and rollback path
- Monitoring and operational evidence

Repo-aligned maturity example

Repo activity

The repository already spans multiple maturity stages: exploratory notebooks, serialized artifacts, Flask APIs, Streamlit dashboards, and CI/CD tutorials. This makes it a strong teaching example for the transition from experimentation to operations.

How this repository illustrates MLOps

- ‘notebooks/‘ shows early-stage exploration and model building.
- ‘pasteurization/‘ and ‘streamlit/‘ show how behavior becomes a service or interface.
- ‘tutorials/Docker‘, ‘tutorials/GitHubActions‘, and ‘tutorials/Jenkins‘ show operationalization paths.
- Monitoring examples in ‘App_Stream.py‘ connect deployed behavior to feedback and drift response.

From This Class to the Next

What stays with us

- MLOps connects experimentation, packaging, release, monitoring, and ownership into one lifecycle.
- The hard part is not only training a model but keeping it reliable after handoff.
- Operational maturity comes from repeatability, observability, and clear responsibilities.

Project use

This is the lens teams can now use to evaluate whether their project behaves like an engineered system or only like an isolated analysis artifact.

Next connection

Class 27 zooms into one MLOps capability in depth: continuous training and the automation of the retrain-and-redeploy loop.

Course thread: The lifecycle view we just built has a training-and-retraining box in it; the next class asks what it actually takes to operationalize that box.

Papers and materials

Papers and materials

Sculley et al., Hidden Technical Debt in Machine Learning Systems; Practical MLOps; Designing Machine Learning Systems; Google Cloud MLOps guidance; MLOps best practices overview; ML observability materials from Evidently.